



Figure 2: Pingdom1



Figure 3: Pingdom2

In our case, after installing Boost, Pingdom reported 4.86 seconds. This is because Boost caches the content on the disk and Drupal doesn't have to generate the pages every time a request is received.

If your Drupal site has modules that set cookies or sessions like QuickTabs, Recaptcha, SimpleNews, Webforms, etc, you should remember that they disable caching. So it's a trade-off between whether you want to retain such modules or opt for speed. You might want to look for alternatives to these modules or, better still, roll out your own module.

Using Boost does have its side effects. If you set the cache expiry to say, four weeks, and you make changes to your site, these will not be visible to the visitor till such time the new page is generated. You could schedule the Boost Crawler to generate pages at pre-determined intervals if your pages are undergoing changes at a frequency that doesn't warrant disabling Boost. On our sites, we have disabled the crawler because it had its overheads; instead, we make it a point to delete the Boost cached pages every time we make changes.

Remove references to missing assets

The 404/410 responses that Pingdom listed were for images that were missing. We removed these references in our theme template files and Pingdom reported healthy response times.

From Amsterdam, Pingdom reported a load time of 2.27 seconds, and 1.26 seconds from Dallas. We achieved the acceptable performance figures and hence stopped further tweaking and tuning. However, for two other sites where we have over 800 unique visitors and at least a 100 logged in sessions every day, we had to implement other measures. They have been listed in the order that they were implemented.

Increase PHP memory limit

One of the methods to improve PHP performance is to increase the memory available to it by specifying a directive in *php.ini*. On Centos machines the file is located at */etc/php*.

ini while on Ubuntu it is found at */etc/php5/apache2/*. Add the following line:

```
memory_limit = 64M ; Maximum amount of memory a script may consume (64MB)
```

To specify unlimited memory, use a value of *'-1'*.

If you are unable to access the *php.ini*, you can specify the following directive in the *.htaccess*:

```
php_value memory_limit 64M
```

Alternatively, you can also specify the following directive in your *settings.php* file. This will affect only the site that uses this file:

```
ini_set('memory_limit', '64M');
```

Implement a PHP op-code caching mechanism called Alternative PHP Cache (APC).

All our Drupal servers have APC installed. APC is an op-code cache. It caches compiled PHP code and hence improves response times. To get your Drupal installation to use APC, you will have to install the APC binaries either using your Linux distribution's package manager or using PECL. After installing it, add the following line to */etc/php5/apache2/php.ini*.

```
extension=apc.so
apc.enabled=1
apc.shm_segments=1
apc.shm_size=64
apc.stat=0
apc.ttl=0
```

Then restart Apache and check *php.ini* in the Drupal status page to see if APC is installed. Check if the shared memory size for APC is sufficient for your page size. The default *apc.shm_size* parameter is set to 32 MB; increase it to 64 MB (you can go up to 96 MB).

Implement a memory based caching mechanism called Memcached

Memcached is a distributed object caching system. We install Memcached on all our Drupal servers. You can install the Memcached binaries using your Linux distribution's package manager. Use PECL to install the Memcached PHP extension, and then add the following lines to */etc/php5/conf.d/memcache.ini*

```
extension=memcache.so
memcache.hash_strategy="consistent"
```

If you have fiddled with the iptables while setting up your VPS, modify the rules to enable port 11211—the port